根据《Phase 2：体系结构设计》任务要求，本文档包含 CampusHub 校园互助平台的完整类图设计以及基于 SOLID 原则的审查结果。类图由 AI 生成，基于已选定的分层单体架构，涵盖用户与认证、需求管理、订单流程、评价信用、社区论坛、通知、管理后台 7 个核心模块的实体类、服务类、仓库接口及其关系。审查部分逐条对照 SOLID 检查清单，标注 AI 设计中的违反项并给出修正方案，最后总结四个核心问题及共性问题。以下为完整内容。

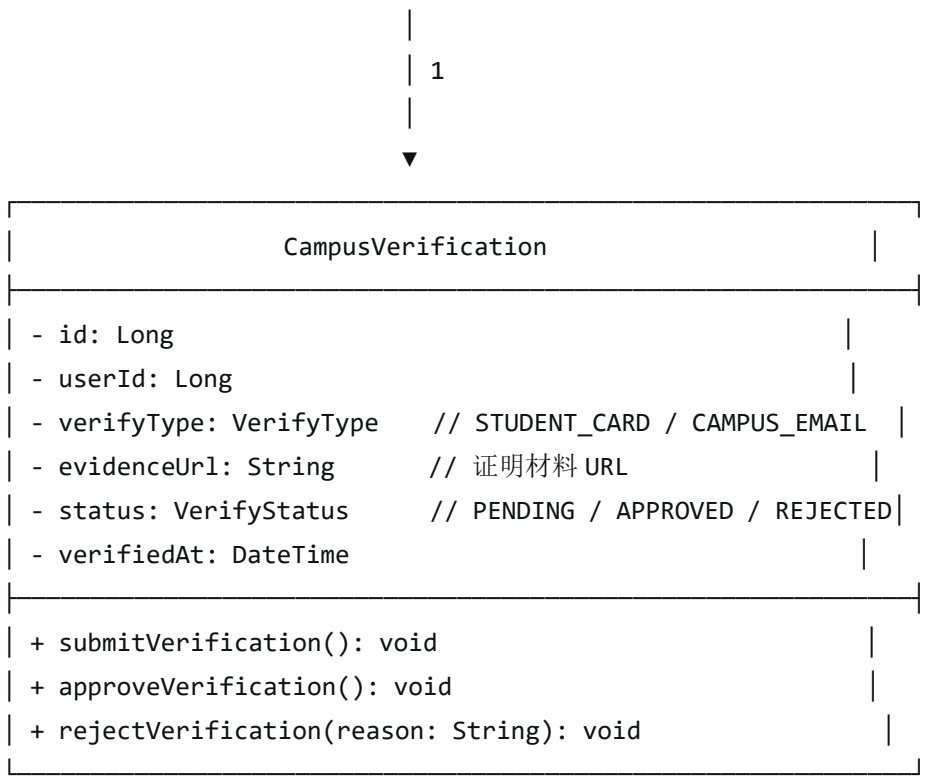
一、CampusHub 系统类图

1. 核心实体类（数据层 / ORM 模型）

用户与认证模块

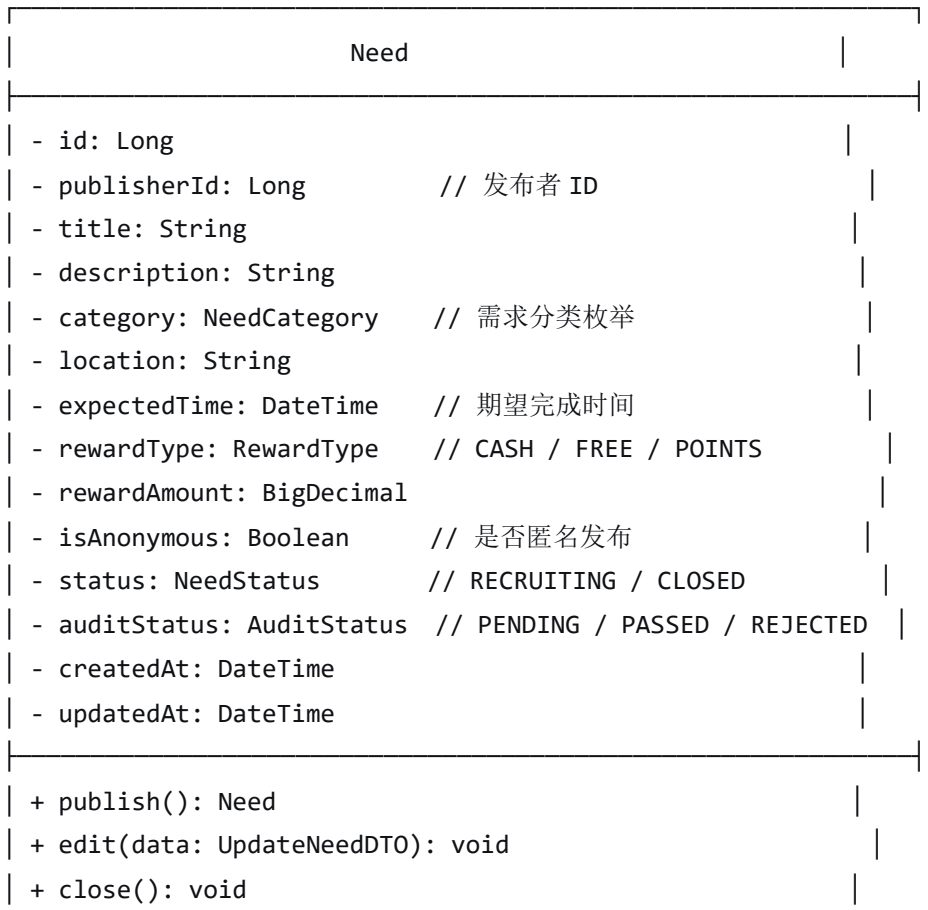
text

```
┌──────────────────────────────────────────────────────────────┐
│                        User                                  │
├──────────────────────────────────────────────────────────────┤
│ - id: Long                                                   │
│ - studentId: String          // 学号                         │
│ - email: String              // 校园邮箱                     │
│ - passwordHash: String       // 加密密码                     │
│ - nickname: String           // 昵称                         │
│ - avatar: String             // 头像 URL                     │
│ - college: String            // 学院                         │
│ - contactInfo: String        // 联系方式                     │
│ - currentRole: Role          // 当前角色: DEMANDER/SERVER    │
│ - createdAt: DateTime                                        │
│ - updatedAt: DateTime                                        │
├──────────────────────────────────────────────────────────────┤
│ + register(): User                                           │
│ + login(): String            // 返回 token                   │
│ + resetPassword(): void                                      │
│ + switchRole(role: Role): void                               │
│ + updateProfile(data: UpdateProfileDTO): void                │
└──────────────────────────────────────────────────────────────┘
```


1
CampusVerification
- id: Long
- userId: Long
- verifyType: VerifyType // STUDENT_CARD / CAMPUS_EMAIL
- evidenceUrl: String // 证明材料 URL
- status: VerifyStatus // PENDING / APPROVED / REJECTED
- verifiedAt: DateTime
+ submitVerification(): void
+ approveVerification(): void
+ rejectVerification(reason: String): void

需求管理模块

text


Need
- id: Long
- publisherId: Long // 发布者 ID
- title: String
- description: String
- category: NeedCategory // 需求分类枚举
- location: String
- expectedTime: DateTime // 期望完成时间
- rewardType: RewardType // CASH / FREE / POINTS
- rewardAmount: BigDecimal
- isAnonymous: Boolean // 是否匿名发布
- status: NeedStatus // RECRUITING / CLOSED
- auditStatus: AuditStatus // PENDING / PASSED / REJECTED
- createdAt: DateTime
- updatedAt: DateTime
+ publish(): Need
+ edit(data: UpdateNeedDTO): void
+ close(): void

```
| + audit(): void             // 敏感词审核                  |
| + getAnonymousDisplayName(): String                        |
└────────────────────────────────────────────────────────────┘

<<enumeration>>              <<enumeration>>        <<enumeration>>
    NeedCategory                 RewardType             NeedStatus
_____________________     ____________________     ____________________

EXPRESS_DELIVERY          CASH                     RECRUITING
STUDY_TUTORING            FREE                     CLOSED
SECOND_HAND_TRADE         POINTS
LOST_AND_FOUND
CARPOOL
ACTIVITY_PARTNER
CONSULTATION_QA
TREE_HOLE
```

订单流程模块

text

```
┌────────────────────────────────────────────────────────────┐
|                        Order                               |
├────────────────────────────────────────────────────────────┤
| - id: Long                                                 |
| - needId: Long              // 关联需求 ID                  |
| - demanderId: Long          // 需求方 ID                    |
| - serverId: Long            // 服务方 ID                    |
| - status: OrderStatus       // 订单状态枚举                 |
| - serverMessage: String     // 服务方附言                   |
| - rejectReason: String      // 拒绝原因                     |
| - fundsStatus: FundsStatus // 资金托管状态                  |
| - completedAt: DateTime                                    |
| - createdAt: DateTime                                      |
| - updatedAt: DateTime                                      |
├────────────────────────────────────────────────────────────┤
| + apply(serverId: Long, message: String): void             |
| + selectServer(serverId: Long): void                       |
| + updateStatusToWaitConfirm(): void                        |
| + confirmComplete(): void                                  |
| + rejectComplete(reason: String): void                     |
| + checkDuplicateApply(): Boolean                           |
| + checkSelfApply(): Boolean                                |
└────────────────────────────────────────────────────────────┘
```

```
<<enumeration>>
   OrderStatus
————————————————

IN_PROGRESS
WAITING_CONFIRM
COMPLETED
REJECTED
DISPUTED

<<enumeration>>
   FundsStatus
————————————————

NOT_REQUIRED
PENDING_ESCROW
HELD
RELEASED
```

评价与信用模块

text

```
┌──────────────────────────────────────────────────────────────┐
│                        Review                                │
├──────────────────────────────────────────────────────────────┤
│ - id: Long                                                   │
│ - orderId: Long                                              │
│ - reviewerId: Long          // 评价者 ID                      │
│ - targetUserId: Long        // 被评价者 ID                    │
│ - rating: Integer           // 星级 1-5                       │
│ - tags: List<String>        // 评价标签                       │
│ - content: String           // 文字评价                       │
│ - isAnonymous: Boolean      // 匿名互评                       │
│ - createdAt: DateTime                                        │
├──────────────────────────────────────────────────────────────┤
│ + submitReview(): Review                                     │
│ + getAnonymousReview(): ReviewVO                             │
└──────────────────────────────────────────────────────────────┘
                              │
                              │ 影响
                              ▼
┌──────────────────────────────────────────────────────────────┐
│                        Credit                                │
├──────────────────────────────────────────────────────────────┤
│ - userId: Long                                               │
```

```
| - baseScore: Integer        // 60 基础分                    |
| - verificationBonus: Integer // 认证加分                    |
| - goodReviewRate: Float     // 好评率                       |
| - totalScore: Integer       // 综合信用分                   |
| - lastUpdated: DateTime                                     |
├─────────────────────────────────────────────────────────────┤
| + calculateScore(): Integer                                 |
| + updateAfterReview(): void                                 |
| + manualAdjust(score: Integer, reason: String): void        |
└─────────────────────────────────────────────────────────────┘
```

社区论坛模块

text

```
┌─────────────────────────────────────────────────────────────┐
|                      Post                                   |
├─────────────────────────────────────────────────────────────┤
| - id: Long                                                  |
| - authorId: Long                                            |
| - title: String                                             |
| - content: String                                           |
| - board: BoardType          // 分区                         |
| - auditStatus: AuditStatus  // 审核状态                      |
| - viewCount: Integer                                        |
| - createdAt: DateTime                                       |
| - updatedAt: DateTime                                       |
├─────────────────────────────────────────────────────────────┤
| + publish(): Post                                           |
| + edit(): void                                              |
| + delete(): void                                            |
| + filterSensitiveWords(): void                              |
└─────────────────────────────────────────────────────────────┘
```

```
<<enumeration>>
    BoardType
_____________________

CAMPUS_NEWS
CLUB_ACTIVITY
LOST_AND_FOUND
FREE_DISCUSSION
```

通知模块

text

```
┌──────────────────────────────────────────────────────────┐
│                      Notification                        │
├──────────────────────────────────────────────────────────┤
│ - id: Long                                               │
│ - recipientId: Long         // 接收者 ID                  │
│ - type: NotifyType          // 通知类型枚举               │
│ - title: String                                          │
│ - content: String                                        │
│ - relatedId: Long           // 关联业务 ID                │
│ - isRead: Boolean                                        │
│ - createdAt: DateTime                                    │
├──────────────────────────────────────────────────────────┤
│ + send(): void                                           │
│ + markAsRead(): void                                     │
│ + getUnreadCount(userId: Long): Integer                  │
└──────────────────────────────────────────────────────────┘
```

```
<<enumeration>>
    NotifyType
————————————————————

APPLY_RECEIVED
STATUS_CHANGED
REVIEW_RECEIVED
FORUM_INTERACTION
SYSTEM_ANNOUNCEMENT
```

管理后台相关

text

```
┌──────────────────────────────────────────────────────────┐
│                    AdminOperation                        │
├──────────────────────────────────────────────────────────┤
│ - id: Long                                               │
│ - adminId: Long                                          │
│ - operationType: OperationType                           │
│ - targetType: TargetType     // CONTENT / USER           │
│ - targetId: Long                                         │
│ - reason: String                                         │
│ - createdAt: DateTime                                    │
├──────────────────────────────────────────────────────────┤
│ + logOperation(): void                                   │
└──────────────────────────────────────────────────────────┘
```

```
<<enumeration>>              <<enumeration>>
  OperationType                TargetType
_____________________    ____________________

BLOCK                      CONTENT
UNBLOCK                    USER
DELETE
APPROVE
```

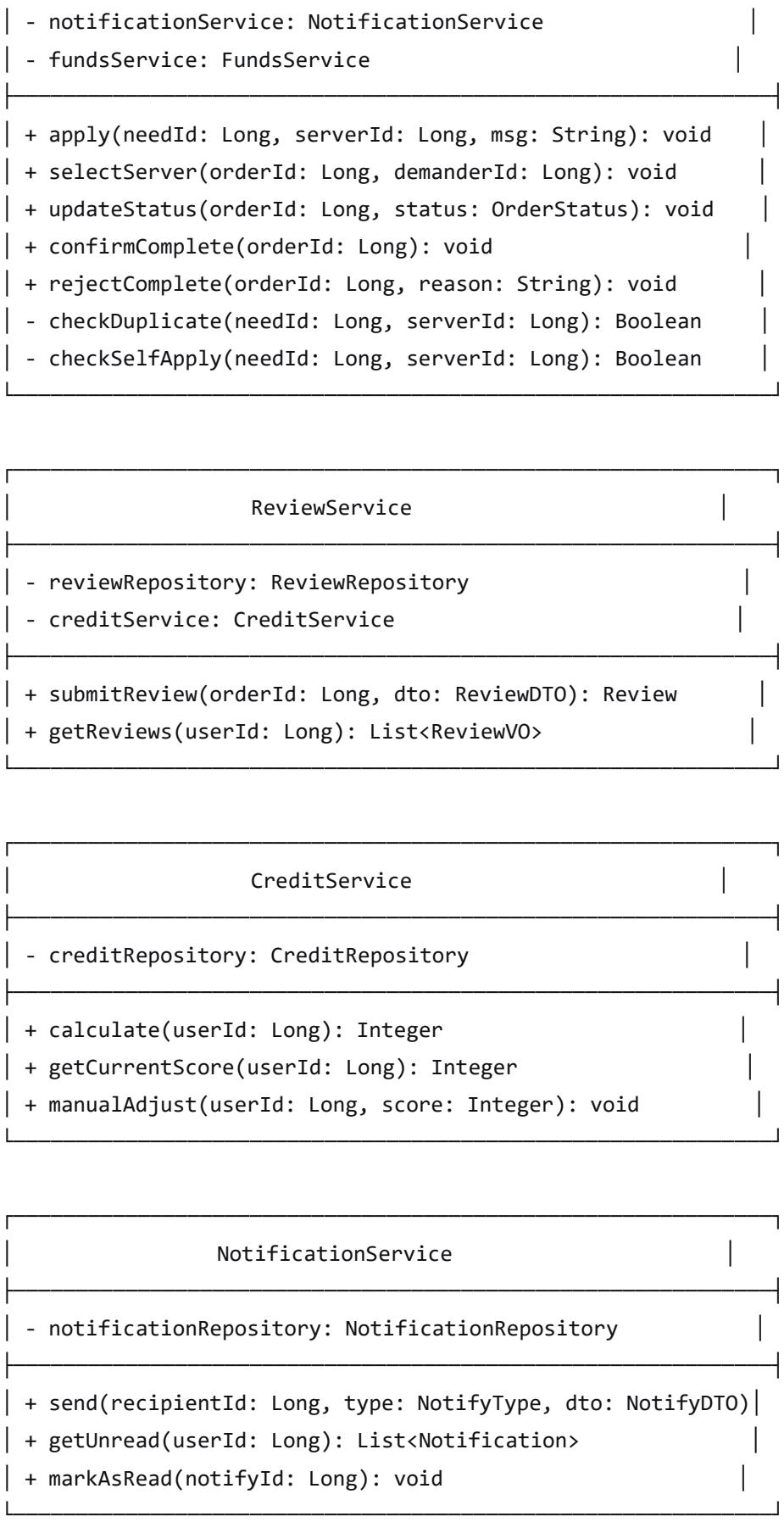
2. 服务层类（业务逻辑层）

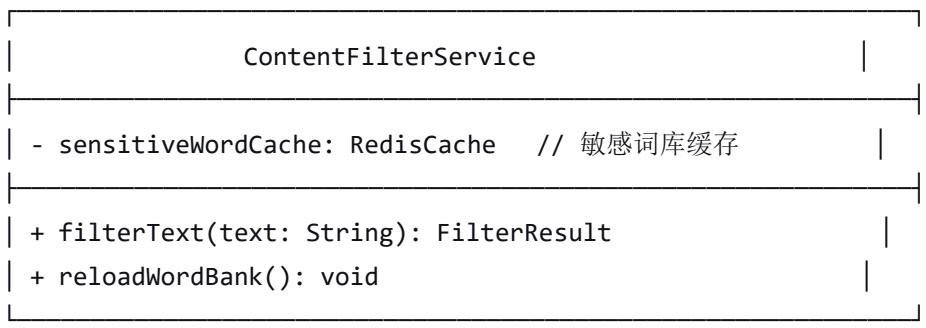
text

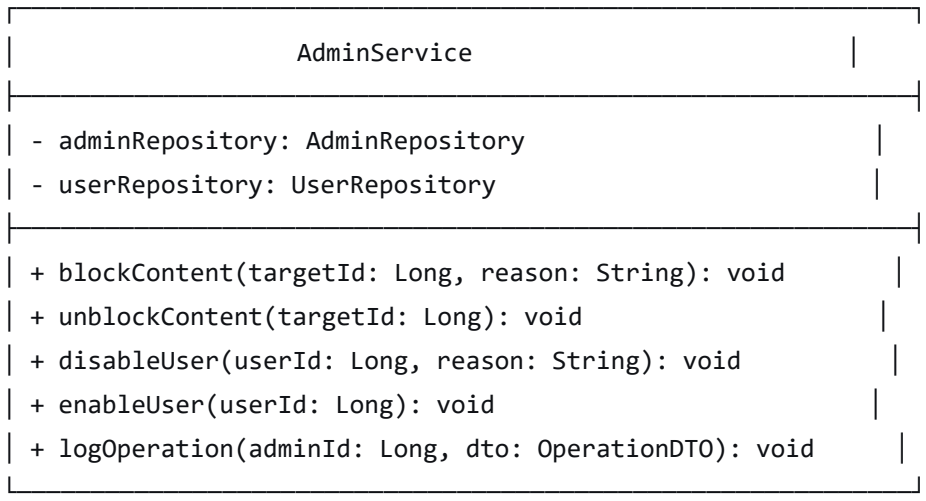
```
┌──────────────────────────────────────────────────────────────┐
│                      UserService                             │
├──────────────────────────────────────────────────────────────┤
│ - userRepository: UserRepository                             │
│ - verificationService: VerificationService                   │
├──────────────────────────────────────────────────────────────┤
│ + register(dto: RegisterDTO): User                           │
│ + login(dto: LoginDTO): AuthToken                            │
│ + verifyCampus(dto: VerifyDTO): void                         │
│ + updateProfile(userId: Long, dto: ProfileDTO): void         │
│ + switchRole(userId: Long, role: Role): void                 │
└──────────────────────────────────────────────────────────────┘

┌──────────────────────────────────────────────────────────────┐
│                      NeedService                             │
├──────────────────────────────────────────────────────────────┤
│ - needRepository: NeedRepository                             │
│ - contentFilterService: ContentFilterService                 │
├──────────────────────────────────────────────────────────────┤
│ + publishNeed(userId: Long, dto: NeedDTO): Need              │
│ + editNeed(needId: Long, dto: NeedDTO): void                 │
│ + closeNeed(needId: Long): void                              │
│ + listNeeds(filter: NeedFilter): Page<Need>                  │
│ + filterByKeyWords(): void                                   │
└──────────────────────────────────────────────────────────────┘

┌──────────────────────────────────────────────────────────────┐
│                      OrderService                            │
├──────────────────────────────────────────────────────────────┤
│ - orderRepository: OrderRepository                           │
```


- notificationService: NotificationService
- fundsService: FundsService
+ apply(needId: Long, serverId: Long, msg: String): void
+ selectServer(orderId: Long, demanderId: Long): void
+ updateStatus(orderId: Long, status: OrderStatus): void
+ confirmComplete(orderId: Long): void
+ rejectComplete(orderId: Long, reason: String): void
- checkDuplicate(needId: Long, serverId: Long): Boolean
- checkSelfApply(needId: Long, serverId: Long): Boolean
ReviewService
- reviewRepository: ReviewRepository
- creditService: CreditService
+ submitReview(orderId: Long, dto: ReviewDTO): Review
+ getReviews(userId: Long): List<ReviewVO>
CreditService
- creditRepository: CreditRepository
+ calculate(userId: Long): Integer
+ getCurrentScore(userId: Long): Integer
+ manualAdjust(userId: Long, score: Integer): void
NotificationService
- notificationRepository: NotificationRepository
+ send(recipientId: Long, type: NotifyType, dto: NotifyDTO)
+ getUnread(userId: Long): List<Notification>
+ markAsRead(notifyId: Long): void





3. 数据访问层（Repository 接口）

```
+ findByBoard()          + findByAdminId()
+ findByAuthorId()       + findByTarget()
+ save(Post): Post       + save(Operation)
```

4. 依赖关系简图

```
text
UserService ———> User, CampusVerification
NeedService ———> Need, ContentFilterService
OrderService ——> Order, NotificationService, FundsService
ReviewService —> Review, CreditService
CreditService —> Credit
NotificationService —> Notification
AdminService ———> AdminOperation, User
```

SOLID 原则	检查问题	AI设计是否违反	违反说明	修正方案
S - 单一职责	有没有类承担了过多职责？	是	Order 实体同时承担订单状态管理、资金托管状态记录、争议原因记录等多项职责；OrderService 中混合状态转移、资金操作、通知触发三种不同职责。	抽离独立的 FundsRecord 实体与 FundsService，通知改为事件发布机制，让 OrderService 仅关注订单自身状态流转。
O - 开闭原则	新增需求类型是否需要修改	是	NeedCategory 和 NotifyType 等枚举硬编码类型列表，新增分类或通知类型必须修改枚举定义及所有	将分类改为数据库配置表 Category，提供管理端维护；订单状态流转改用状态模式，新增状态只需扩展子类。

SOLID 原则	检查问题	AI设计是否违反	违反说明	修正方案
	现有代码？		依赖它的筛选、发送逻辑。	
L - 里氏替换	子类是否可以替换父类使用？	否	当前设计未使用继承体系，所有实体平铺，未定义父类，因此不存在替换问题，符合 MVP 阶段简化策略。	无需修正。若未来引入 Admin extends User 等继承，需确保子类行为与父类契约一致。
I - 接口隔离	有没有接口太“胖”，包含了不需要的方法？	是	NotificationRepository 同时定义读操作（findByRecipientId，getUnreadByUserId）和写操作（save），若只发送通知的模块被迫依赖不需要的查询方法。	拆分为 NotificationWriteRepository 和 NotificationReadRepository；对 AdminOperationRepository 同理拆分。
D - 依赖倒转	高层模块是否直接依赖于低层模块的具体实现？	是	OrderService 直接引用了 NotificationService 和 FundsService 的具体类，而非接口；NeedService 直接依赖 ContentFilterService 的具体实现。	为 NotificationService 和 ContentFilterService 定义抽象接口，通过依赖注入容器将具体实现注入到 OrderService 等高层模块。

审查总结：

AI 生成的初始类图在业务建模上是完整的，但在 SOLID 合规性上存在四个核

心问题：单一职责违反（`Order`/`OrderService` 职责混合）、开闭原则违反（枚举硬编码）、接口隔离违反（仓库接口过胖）、依赖倒转违反（高层直接依赖具体实现）。这四项缺陷反映出 AI 倾向于生成“能跑通流程”的平铺设计，而忽视了模块隔离、可扩展性和松耦合等架构质量属性。其中最严重的是依赖倒转问题，因为它直接违背了我们架构决策中“模块间通过定义好的接口通信”的约束，一旦底层通知或审核实现需要替换，核心业务逻辑将被迫重写。所有修正方案均已在表格中给出，后续编码阶段必须严格按修正后的类结构实施。